

Assignment - 6

Topic : Recurrent Neural Network (RNN)

Q.1 What are sequence learning problem. Explain with example.

⇒

- A sequence learning problem involves processing data where the order of elements matters.
- Unlike traditional machine learning tasks that work with fixed-size inputs, sequence learning deals with variable-length data, where temporal or sequential dependencies are crucial.
- In FCNN and CNN:
 - i) the output at any time step is independent of the previous layer input/output.
 - ii) the input was always of the fixed-length.
- In general, for fully connected neural network, CNN, the output at "t" time step is independent of any of the previous input/output.
- In "Sequence Learning Problem", the "two properties of FCNN and CNN do not hold" and the output at any time step depends on previous input/output and the length of the input is not fixed.
- For e.g., Sentiment analysis
- It's not mandatory to produce an output at each step.
- For example, for sentiment analysis the model

looks at the words in a sentence and gives/predict a final output as positive/negative sentiment conveyed by the sentence.

- This could be considered as output is produced at every time step but the model ignores those outputs and reports only the final output.
- This is also termed as Sequence Classification Problem.

Q.2 Difference between Fully Connected Neural Network and RNN

⇒

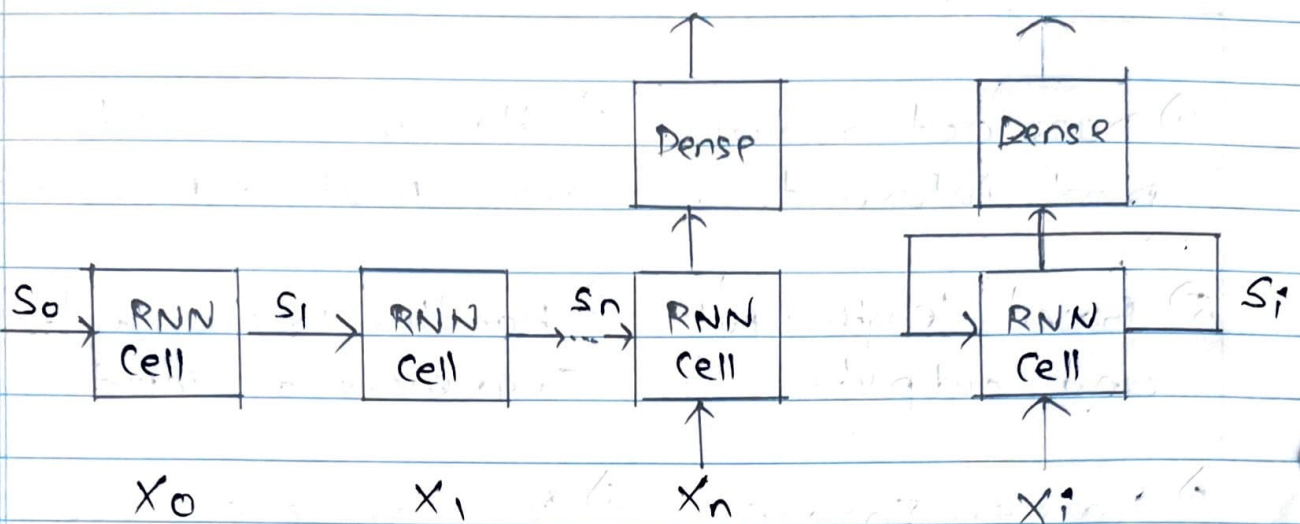
FCNN	RNN
1) Consists of multiple fully connected layers, where each neuron is connected to every neuron in next layer.	1) Has recurrent connections, allowing information to persist across time steps.
2) Cannot handle sequential data effectively as it lacks memory.	2) Designed for sequential data with time dependencies.
3) Different weights for each layer.	3) Shared weight across time steps.

FCNN	RNN
4) Does not retain past information.	4) Has internal state to store past information.
5) Fixed length input and output.	5) Handles variable length sequences as input and output.
6) For e.g., Image classification, traditional ML tasks.	6) For e.g., speech recognition, text generation, etc.

Q.3 Explain RNN with appropriate architecture block diagram.

⇒

- RNN are called recurrent because they perform the same task for every element of a sequence, with the output being dependent on the previous computations.
- RNN shares similarities in input and output structures with other deep learning architectures but differ significantly in how information flows from input to output.
- Unlike traditional deep neural networks, where each dense layer has distinct weight matrices, RNNs use shared weights across time steps, allowing them to remember information over sequences.



- In RNNs, the hidden state (H_i) is calculated for every input X_i , to retain sequential dependencies.
- The computation follows these core formulas:

1) Hidden state calculation :

$$h = \sigma(U \cdot X + W \cdot h_{t-1} + B)$$

Here, h = current hidden state,
 U, W = weight matrices,
 B = Bias

2) Output calculations :

$$y = O(V \cdot h + c)$$

The output y_i is calculated by applying O , an activation function, to the weighted hidden state

3) Overall Function:

$$Y = F(X, h, W, U, V, B, C)$$

This function defines the entire RNN operation, where the state matrix S holds each element S_i , representing the network's state at each time step i .

Q.4 Explain the exploding and vanishing gradient in RNN.

⇒

i) Exploding Gradient

- Exploding gradients are a problem when large error gradient accumulate and results in very large updates to neural network model weights during training.
- A gradient calculates the direction and magnitude during the training of neural network and it is used to teach the network weights in the right direction by the right amount.
- When there is an error gradient, the explosion of components may grow exponentially.
- When large error gradient accumulate, the model may become unstable and unable to learn from training data.

- At an extreme, the values of weights can become so large as to overflow and result in NaN values.

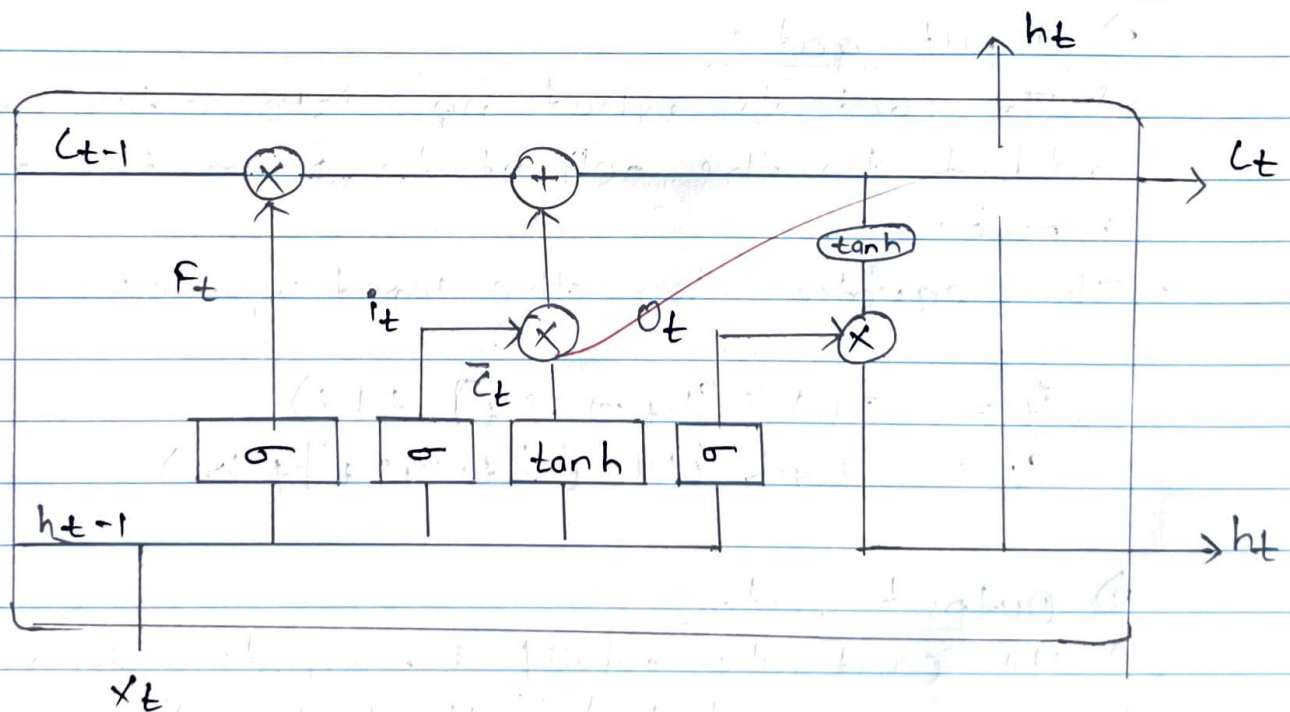
ii) Vanishing Gradient

- Vanishing gradient does not necessarily imply that the gradient vector is all zero.
- It implies that the gradients are minuscule, which could cause the learning to be very low.
- Following are some indications :-
 - a) the model weights could become 0 during training.
 - b) The parameters of higher layer change to a great extent, while the parameters of lower layers barely change.
 - c) The model learns at a particular slow pace and the training could stagnate at a very early phase after only a few iterations.
- When making use of back-propagation the goal is to calculate the error which is actually found by finding out the difference between the actual output and the model output and raising that to a power of 2.

Q.B Explain the LSTM architecture in detail

⇒

- Long Short-Term Memory is an enhanced version of RNN.
- LSTM architecture involves the memory cell which is controlled by three gates: input, forget and output gate.
- These gates decide what information to add to, remove from and output from the memory cell.



1) Forget Gate

⇒ The information that is no longer useful in the cell state is removed with the forget gate.

- Two input x_t and h_{t-1} are fed to the gate and multiplied with weight matrices followed

by the addition of bias.

- The resultant is passed through an activation function which gives a binary output.
- If for a particular cell state the output is 0, the piece of information is forgotten and for output 1, the information is retained for future use.
- The equation is-

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

2) Input gate

- ⇒ It controls what new information will be added to the cell state from the current input.
- The equation for the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

$$\hat{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

3) Output gate

- ⇒ It controls which information to reveal from the updated cell state to the output in a single time step.
- The equation is-

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

Q.5 Explain different gates and their functions in an LSTM cell.

⇒

1) Forget Gate

⇒ Decides which past information should be forgotten from the cell state.

— The equation:

$$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$$

— Components:

Sigmoid activation (σ): Produces values between 0 and 1

$f_t \approx 0 \rightarrow$ Forget completely

$f_t \approx 1 \rightarrow$ Keep fully.

— For e.g., language model.

2) Input Gate

⇒ Decides what new information should be stored in the cell state.

— The equation:

$$i_t = \sigma(W_i [h_{t-1}, x_t] + b_i)$$

$$\bar{c}_t = \tanh(W_c [h_{t-1}, x_t] + b_c)$$

— Here,

σ = Decides which value to update

$\tanh$ = creates new candidate values

— Final update: $c_t = f_t \cdot c_{t-1} + i_t \cdot \bar{c}_t$

3) Output Gate

⇒ Controls what part of the cell state is passed as the output.

→ The equation:

$$O_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = O_t \cdot \tanh(C_t)$$

→ For e.g., text generation

(A)

~~analysis~~